

Lesson 51 Practice: Answers and Rationales

JKSSB Computer Awareness

October 8, 2026

Provenance: All 20 items are original JKSSB-pattern questions (*[Original]*); none reproduces an official paper item.

- Q1. A.** OOP stands for Object-Oriented Programming. B, C, and D are not the accepted expansion of the abbreviation.
- Q2. A.** LIFO stands for Last In, First Out. B, C, and D are not the correct full forms.
- Q3. B.** Machine language is the only language the CPU executes directly. Assembly (A) must be assembled, a high-level language (C) must be compiled or interpreted, and pseudocode (D) is not executable at all.
- Q4. A.** Short codes such as ADD and MOV are called mnemonics. Keywords (B) are words reserved by a high-level language; variables (C) and functions (D) are program elements, not instruction codes.
- Q5. C.** An assembler translates assembly language into machine code. A compiler (A) and an interpreter (B) translate high-level languages; a linker (D) combines object files but does not translate assembly source.
- Q6. B.** A compiler translates the whole program at once, while an interpreter translates and executes it line by line. A reverses the two; C wrongly claims both work on the whole program; D is false because a compiler does translate the source.
- Q7. B.** Source code is the program written by the programmer in a high-level or assembly language and is human-readable. A describes machine code, C describes object code, and D describes firmware.
- Q8. B.** The compiler's machine-readable output is called object code. Source code (A) is the human-written input, pseudocode (C) is a design notation, and natural-language code (D) is not a technical term.
- Q9. B.** An algorithm is a finite, step-by-step procedure to solve a problem. A describes a flowchart, C describes machine code, and D describes a debugger.
- Q10. C.** A decision is shown by a diamond (rhombus). A rectangle (A) is a process, a parallelogram (B) is input/output, and an oval (D) is a terminator for start or end.
- Q11. B.** Finding and correcting errors is debugging. Compiling (A) translates source code, linking (C) joins object files, and encoding (D) is unrelated to error correction.

- Q12. B.** A program that compiles and runs but gives a wrong result has a logical error. A syntax error (A) would stop compilation, a runtime error (C) occurs during execution, and a translation error (D) is not a standard category.
- Q13. B.** A class is a blueprint or template from which objects are created. A reverses the relation (an object is an instance of a class), C confuses a class with storage, and D confuses it with a machine instruction.
- Q14. C.** “One name, many forms” describes polymorphism. Encapsulation (A) binds data and methods, inheritance (B) reuses a parent class, and abstraction (D) hides implementation detail.
- Q15. B.** A stack follows LIFO (Last In, First Out). FIFO (A) is the queue order; random (C) and sorted (D) are not the stack rule.
- Q16. B.** Stack insertion and deletion are called push and pop. Enqueue and dequeue (A) belong to a queue; insert/remove (C) and add/delete (D) are generic terms, not the stack operations.
- Q17. B.** A queue follows FIFO (First In, First Out). LIFO (A) is the stack order; random (C) and reverse (D) are not queue rules.
- Q18. C.** A root node with child nodes describes a tree. An array (A) is a flat indexed collection, a stack (B) is linear LIFO, and a queue (D) is linear FIFO.
- Q19. C.** A compiler does not translate and execute line by line; that is the interpreter’s method, so C is NOT true. Translating the whole program at once (A), producing object code (B), and reporting errors after the full translation (D) are all true of a compiler.
- Q20. C.** Statements I and III are correct: an array stores same-type elements in contiguous memory, and a stack is LIFO. II is false because a linked list does not store its elements contiguously; it links separate nodes by pointers. A includes false II, B includes false II, and D includes false II.